🧩 1. What is Algorithm Complexity?

When you write a program (or algorithm), two main questions arise:

1. **How fast is it?** → ⏱️ **Time Complexity**
2. **How much memory does it use?** → 💾 **Space Complexity**

These are collectively called **Algorithm Complexities** — they help us **compare** different algorithms and choose the best one for a problem.

⚙️ 2. TIME COMPLEXITY

👉 Definition:

Time Complexity measures **how the running time of an algorithm increases** with the **input size (n)**.

It is not about **exact time in seconds**, but rather how the **number of operations grows** as n increases.

🧠 Example 1: Constant Time – O(1)

```
def get_first_element(arr):
    return arr[0]
```

✅ This function always takes **1 step**, no matter how large the array is.

👉 **Time Complexity = O(1)**

Input Size (n)	Operations	Big-O
10	1	O(1)
1000	1	O(1)

🧠 Example 2: Linear Time – O(n)

```
def print_elements(arr):
    for item in arr:
```

```
    print(item)
```

Here, time increases **linearly** with array size.

Input Size (n)	Operations	Big-O
10	10	O(n)
1000	1000	O(n)

👉 **Time Complexity = O(n)**

🧠 **Example 3: Quadratic Time – O(n²)**

```
def print_pairs(arr):
  for i in arr:
    for j in arr:
      print(i, j)
```

Each element pairs with all others → n × n = n² operations.

Input Size (n)	Operations	Big-O
10	100	$O(n^2)$
1000	1,000,000	$O(n^2)$

👉 **Used in:** Nested loops like Bubble Sort, Insertion Sort.

🧠 **Example 4: Logarithmic Time – O(log n)**

```
def binary_search(arr, target):
  low, high = 0, len(arr) - 1
  while low <= high:
    mid = (low + high) // 2
    if arr[mid] == target:
      return mid
```

```
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
  return -1
```

Each step cuts the data in half → **$\log_2(n)$** steps.

Input Size (n)	Operations	Big-O
8	3	O(log n)
16	4	O(log n)

👉 **Used in:** Binary Search, Balanced Trees, Divide & Conquer Algorithms.

🧠 **Example 5: Exponential Time – $O(2^n)$**

```
def fibonacci(n):
  if n <= 1:
    return n
  return fibonacci(n-1) + fibonacci(n-2)
```

Each call spawns two more → total ≈ 2^n calls.

🚨 Extremely inefficient for large n.

n	Operations	Big-O
5	32	$O(2^n)$
10	1024	$O(2^n)$

👉 **Used in:** Recursive problems without memoization (like brute-force subset generation).

📊 **3. Common Time Complexities**

Complexity	Name	Example
O(1)	Constant	Accessing element by index
O(log n)	Logarithmic	Binary search
O(n)	Linear	Traversing array
O(n log n)	Linearithmic	Merge sort, Quick sort
O(n^2)	Quadratic	Nested loops
O(2^n)	Exponential	Recursive Fibonacci
O(n!)	Factorial	Generating permutations

💾 4. SPACE COMPLEXITY

👉 Definition:

Space Complexity measures **the total memory required by the algorithm**, including:

1. **Input space**
2. **Auxiliary space** (temporary variables, recursion stack, etc.)

🧠 Example 1: O(1) Space

```
def sum_array(arr):
    total = 0
    for num in arr:
        total += num
    return total
```

Only total variable used → constant memory.

👉 Space Complexity = O(1)

🧠 Example 2: O(n) Space

```
def copy_array(arr):
    new_arr = []
    for num in arr:
        new_arr.append(num)
    return new_arr
```

A new array of size n created →

👉 **Space Complexity = O(n)**

🧠 **Example 3: Recursion Space**

Each recursive call adds to the **call stack**.

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n-1)
```

Here, there are n calls → **Space Complexity = O(n)** (due to recursion stack).

⚖️ **5. Relation Between Time and Space**

Often, optimizing for **time** increases **space** (and vice versa):

Algorithm	**Time**	**Space**
Recursive Fibonacci	$O(2^n)$	O(n)
DP Fibonacci	O(n)	O(n)
Iterative Fibonacci	O(n)	O(1)

👉 Trade-off: Sometimes you store extra data (using space) to compute faster.

🧩 **6. Real-Life Analogy**

Situation	Time Complexity	Space Complexity
Finding a name in phone book (Binary Search)	O(log n)	O(1)
Finding a name in random list	O(n)	O(1)
Memorizing results to avoid rework (Dynamic Programming)	↓ Time	↑ Space

---

🔍 7. Big-O, Big-Ω, and Big-Θ Notations

Notation	Meaning	Example
O(f(n))	Worst case (upper bound)	Bubble sort → $O(n^2)$
Ω(f(n))	Best case (lower bound)	Bubble sort → Ω(n)
Θ(f(n))	Average case (tight bound)	Merge sort → Θ(n log n)

🚀 8. Practical Interview Tip

When analyzing an algorithm:

1. Count **loops** → gives time growth.
2. Count **extra data structures** → gives space growth.
3. Use **asymptotic notation** (ignore constants & small terms).

Example:

```
for i in range(n):        # O(n)
    for j in range(5):    # O(5) -> O(1)
        print(i, j)
```

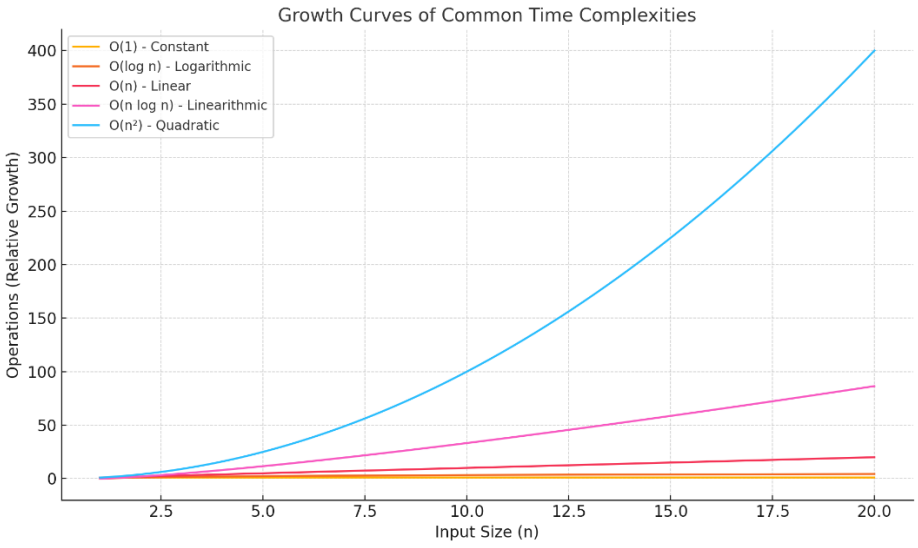
Total = n × 1 = O(n) ✅

🧮 9. Summary Table

Type	Example	Time	Space
Constant	Access element	O(1)	O(1)
Linear	Traverse array	O(n)	O(1)
Quadratic	Nested loop	$O(n^2)$	O(1)
Logarithmic	Binary Search	O(log n)	O(1)
Exponential	Recursive Fibonacci	$O(2^n)$	O(n)

✅ 10. Key Takeaways

- **Time Complexity** → How *fast* the algorithm runs.
- **Space Complexity** → How *much memory* it uses.
- Always think in terms of **input size (n)**.
- Use **Big-O Notation** to express scalability.
- Optimize for **balance** — fast enough and memory efficient.

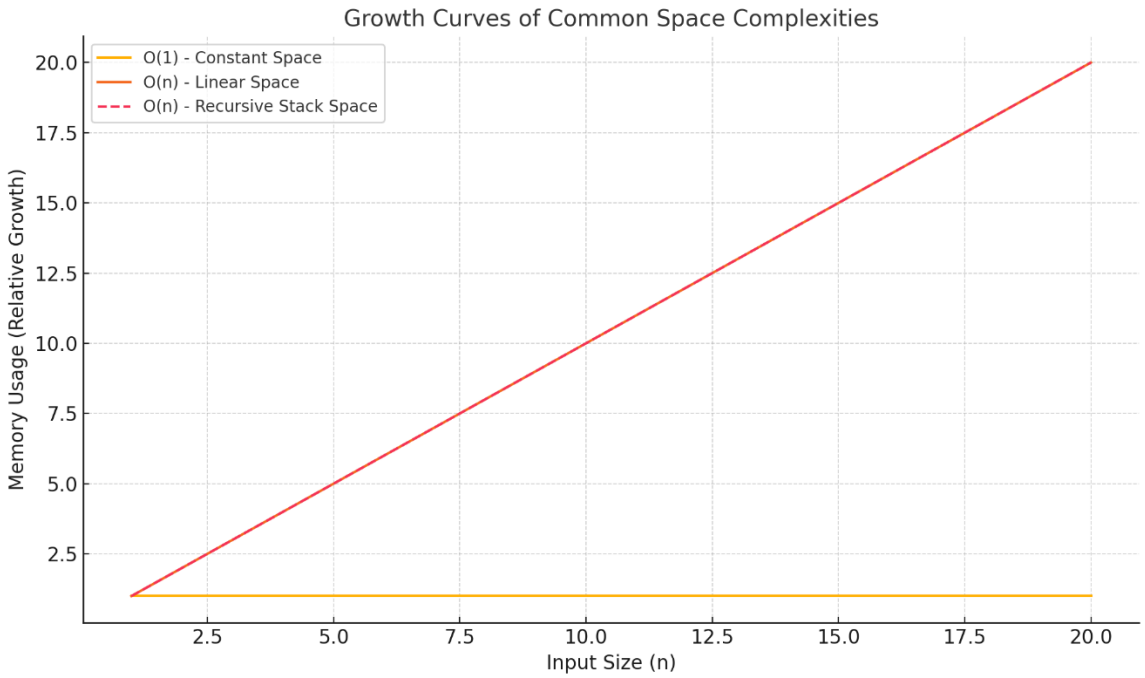


Here's the visual comparison 👆 showing how different **time complexities** grow as input size increases:

- 🟩 **O(1)** stays flat (constant time)

- 🟦 **O(log n)** grows very slowly
- 🟧 **O(n)** increases linearly
- 🟪 **O(n log n)** grows moderately
- 🔴 **O(n²)** grows rapidly

This helps you visually understand why algorithms with **O(log n)** or **O(n)** are far more efficient for large inputs than **O(n²)** or higher.



Here's the visual chart 👆 showing **space complexity growth**:

- 🟩 **O(1)** → Constant memory (e.g., single variable).
- 🟦 **O(n)** → Linear memory (e.g., array copy, dynamic programming).
- 🟪 **O(n)** (dashed line) → Recursive stack memory usage.

This helps you see that algorithms with recursion or additional data storage grow linearly in memory use — while constant-space algorithms remain efficient.